

Exercises 01

DBMSs, PostgreSQL, Catalogs

[\[Show with no answers\]](#) [\[Show with all answers\]](#)

Some of these questions require you to look beyond the Week 01 lecture material for answers. Some of the questions preempt material that we'll be looking at over the next few weeks. To answer some questions, you may need to look at the PostgreSQL documentation or at the texts for the course ... or, of course, you could simply reveal the answers, but where's the fun in that?

1. List some of the major issues that a relational database management system needs to concern itself with.

[\[show answer\]](#)

2. Give an overview of the major stages in answering an SQL query in a relational database management system. For each step, describe its inputs and outputs and give a brief description of what it does.

[\[show answer\]](#)

3. PostgreSQL is an “object-relational database management system”. What are the differences between PostgreSQL and a “conventional” relational database management system (such as Oracle)?

[\[show answer\]](#)

4. A PostgreSQL installation includes a number of different “scopes”: *databases* (or catalogs), *schemas* (or namespaces), and *tablespaces*. The scopes correspond to notions from the SQL standard. Explain the difference between these and give examples of each.

[\[show answer\]](#)

5. For each of the following command-line arguments to the `psql` command, explain what it does, when it might be useful, and how you might achieve the same effect from within `psql`:

a. `-l`



- b. **-f**
- c. **-a**
- d. **-E**

[\[show answer\]](#)

6. PostgreSQL has two main mechanisms for adding data into a database: the SQL standard **INSERT** statement and the PostgreSQL-specific **COPY** statement. Describe the differences in how these two statement operate. Use the following examples, which insert the same set of tuples, to motivate your explanation:

```
insert into Enrolment(course,student,mark,grade)
    values ('COMP9315', 3312345, 75, 'DN');
insert into Enrolment(course,student,mark,grade)
    values ('COMP9322', 3312345, 80, 'DN');
insert into Enrolment(course,student,mark,grade)
    values ('COMP9315', 3354321, 55, 'PS');

copy Enrolment(course,student,mark,grade) from stdin;
COMP9315      3312345 75      DN
COMP9322      3312345 80      DN
COMP9315      3354321 55      PS
\.
```

[\[show answer\]](#)

7. In **psql**, the **\timing** command turns on a timer that indicates how long each SQL command takes to execute. Consider the following trace of a session asking the several different queries multiple times:

```
unsw=# \timing
Timing is on.
unsw=# select max(id) from students;
      max
-----
 9904944
Time: 112.173 ms
unsw=# select max(id) from students;
      max
-----
 9904944
Time: 0.533 ms
unsw=# select max(id) from students;
      max
-----
 9904944
```



```
Time: 0.484 ms
unsw=# select count(*) from courses;
count
-----
80319
Time: 132.416 ms
unsw=# select count(*) from courses;
count
-----
80319
Time: 30.438 ms
unsw=# select count(*) from courses;
count
-----
80319
Time: 34.034 ms
unsw=# select max(id) from students;
max
-----
9904944
Time: 0.765 ms
unsw=# select count(*) from enrolments;
count
-----
2816649
Time: 2006.707 ms
unsw=# select count(*) from enrolments;
count
-----
2816649
Time: 1099.993 ms
unsw=# select count(*) from enrolments;
count
-----
2816649
Time: 1109.552 ms
```

Based on the above, suggest answers to the following:

- Why is there such variation in timing between different executions of the same command?
- What timing value should we ascribe to each of the above commands?
- How could we generate reliable timing values?
- What is the accuracy of timing results that we can extract like this?

[\[show answer\]](#)



8. Both the `pg_catalog` schema and the `information_schema` schema contain meta-data describing the content of a database. Why do we need two schemas to do essentially the same task, and how are they related?

[\[show answer\]](#)

9. Cross-table references (foreign keys) in the `pg_catalog` tables are defined in terms of `oid` attributes. However, examination of the the catalog table definitions (either via `\d` in `psql` or via the PostgreSQL documentation) doesn't show an `oid` in any of the lists of table attributes. To see this, try the following commands:

```
$ psql mydb
...
mydb=# \d pg_database
...
mydb=# \d pg_authid
```

Where does the `oid` attribute come from?

Every tuple in PostgreSQL contains some "hidden" attributes, as well as the data attributes that were defined in the table's schema (i.e. its `CREATE TABLE` statement). The tuple header containing these attributes is described in section [54.5 Database Page Layout](#) of the PostgreSQL documentation. All tuples have attributes called `xmin` and `xmax`, used in the implementation of multi-version concurrency control. In fact the `oid` attribute is optional, but all of the `pg_catalog` tables have it. You can see the values of the hidden attributes by explicitly naming the attributes in a query on the table, e.g.

```
select oid,xmin,xmax,* from pg_namespace;
```

In other words, the "hidden" attributes are not part of the SQL `*` which matches *all* attributes in the table.

xxAAxx);?>

10. Write an SQL view to give a list of table names and table oid's from the public namespace in a PostgreSQL database.

[\[show answer\]](#)

11. Using the tables in the `pg_catalog` schema, write a function to determine the location of a table in the filesystem. In other words, provide your own implementation of the built-in function: `pg_relation_filepath(TableName)`. The function should be defined and behave as follows:



```
create function tablePath(tableName text) returns text
as $$ ... $$ language plpgsql;

mydb=# select tablePath('myTable');
          tablepath
-----
PGDATA/base/2895497/2895518
mydb=# select tablePath('ImaginaryTable');
          tablepath
-----
No such table: imaginarytable
```

Start the path string with `PGDATA/base` if the `pg_class.reltablespace` value is 0, otherwise use the value of `pg_tablespace.spclocation` in the corresponding `pg_tablespace` tuple.

[\[show answer\]](#)

12. Write a PL/pgSQL function to give a list of table schemas for all of the tables in the public namespace of a PostgreSQL database. Each table schema is a text string giving the table name and the name of all attributes, in their definition order (given by `pg_attribute.attnum`). You can ignore system attributes (those with `attnum < 0`). Tables should appear in alphabetical order.

The function should have following header:

```
create or replace function tableSchemas() returns setof text ...
```

and is used as follows:

```
uni=# select * from tableschemas();
          tableschemas
-----
assessments(item, student, mark)
courses(id, code, title, uoc, convenor)
enrolments(course, student, mark, grade)
items(id, course, name, maxmark)
people(id, ptype, title, family, given, street, suburb, pcode, gender,
(5 rows)
```

[\[show answer\]](#)

13. Extend the function from the previous question so that attaches a type name to each attribute name. Use the following function to produce the string for each attribute's type:



```

create or replace function typeString(typid oid, typmod integer) return
as $$
declare
    typ text;
begin
    typ := pg_catalog.format_type(typid, typmod);
    if (substr(typ, 1, 17) = 'character varying')
    then
        typ := replace(typ, 'character varying', 'varchar');
    elsif (substr(typ, 1, 9) = 'character')
    then
        typ := replace(typ, 'character', 'char');
    end if;
    return typ;
end;
$$ language plpgsql;

```

The first argument to this function is a `pg_attribute.atttypid` value; the second argument is a `pg_attribute.atttypmod` value. (Look up what these actually represent in the PostgreSQL documentation).

Use the same function header as above, but this time the output should look like (for the first three tables at least):

```

assessments(item:integer, student:integer, mark:integer)
courses(id:integer, code:char(8), title:varchar(50), uoc:integer, conv
enrolments(course:integer, student:integer, mark:integer, grade:char(2

```

[\[show answer\]](#)

14. The following SQL syntax can be used to modify the length of a `varchar` attribute.

```

alter table TableName alter column ColumnName set data type varchar(N);

```

where `N` is the new length.

If PostgreSQL did not support the above syntax, suggest how you might be able to achieve the same effect by manipulating the catalog data.

[\[show answer\]](#)

